

HOSPITAL MANAGEMENT SYSTEM

A DBMS Capstone Project Report submitted in partial fulfillment of the requirements for the
award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

G. SANTHI SWAROOPA – 25B11AI331

V. SAILAJA –25B11AIC41

M. LAXMI VARA PRASAD– 25B11AI756

V. M V SIVA SAI GANESH– 25B11AIC67

Under the esteemed guidance of

Mr. T. Srinivasulu, M.Tech., (Ph.D.)

Assistant Professor



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ADITYA UNIVERSITY

(Formerly Aditya Engineering College(A))

(Formerly Aditya Engineering College (A)) Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh,
India

2026–2027

ADITYA UNIVERSITY

(Formerly Aditya Engineering College(A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



CERTIFICATE

This is to certify that the DBMS Capstone Project report entitled “**HOSPITAL MANAGEMENT SYSTEM**” is being submitted by **G. SANTHI SWAROOPA (Roll No. 25B11AI331)**, **V. SAILAJA (Roll No. 25B11AIC41)**, **M. LAXMI VARA PRASAD (Roll No. 25B11AI756)** and **V. M V SIVA SAI GANESH (Roll No. 25B11AIC67)** in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the Capstone Project for the course Database Management Systems (2501IT05) during the academic year 2026–2027, and records the database design, implementation, and documentation undertaken by the team.

Project Guide

Department

Mr. T. Srinivasulu, M. Tech, (Ph.D.)
M. Tech, Ph.D. Associate professor
Associate Professor & HOD

Head of the

Dr. K. Naga Bhargavi,

External Examiner

DECLARATION

We hereby declare that the DBMS Capstone Project entitled “**HOSPITAL MANAGEMENT SYSTEM**” is a genuine work carried out by us. The database design, SQL implementation, queries and documentation presented in this report are our own work, completed under the guidance of **Mr. T. SRINIVASULU**, Department of **ARTIFICIAL INTELLIGENCE & MACHINE LEARNING**, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associates

G.SANTHI SWAROOPA (25B11AI331)

V. SAILAJA (25B11AIC41)

M.LAXMI VARA PRASAD (25B11AI756)

V. M V SIVA SAI GANESH (25B11AIC67)

Date: 23-09-2026

Place: Surampalem

ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide **Mr. T. SRINIVASULU**, Department of **ARTIFICIAL INTELLIGENCE & MACHINE LEARNING**, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to **Dr. K. NAGA BHARGAVI**, Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We extend our sincere thanks to the Management, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, Dr. M. B. Srinivas, Vice-Chancellor, and Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), for providing an excellent academic environment.

Not to forget, we thank the Teaching and Non-Teaching Staff, our parents and our friends, who have directly or indirectly supported us in completing this project on time.

Project Associates

G.SANTHI SWAROOPA - 25B11AI331

V. SAILAJA - 25B11AIC41

M. LAXMI VARA PRASAD - 25B11AI756

V. M V SIVA SAI GANESH - 25B11AIC67

ABSTRACT

A hospital produces interrelated clinical and administrative data through patient registration, departmental services, doctor consultation, appointment scheduling, ward allocation, treatment recording and billing. If these records are maintained in independent registers or spreadsheets, the same facts are copied repeatedly, updates become inconsistent and reports require time-consuming manual compilation. The proposed Hospital Management System solves these problems through a centralized relational database.

Exactly seven entities—Department, Doctor, Patient, Ward, Appointment, Treatment and Bill—form the database. Patient contact is stored as a single contact attribute in Patient and is not modeled as a separate entity. The conceptual model is translated into relations, normalized up to Third Normal Form and implemented in MySQL 8.0 using primary keys, foreign keys, UNIQUE, NOT NULL, CHECK and DEFAULT constraints. Representative data and seventeen SQL questions demonstrate retrieval, joins, aggregate functions, GROUP BY, HAVING, nested and correlated subqueries, views and set operations. DCL and TCL commands demonstrate access control and safe transaction management. The resulting design reduces redundancy, preserves referential integrity and supports reliable operational and billing reports.



INDEX

Chapter	Title	Page No.
	Certificate	2
	Declaration	3
	Acknowledgement	4
	Abstract	5
1	Introduction	8
2	System Study and Tools	10
3	Database Design	12
4	Database Implementation	21
5	SQL Operations and Queries	27
6	Modules and Functionalities	37
7	Results – Sample Outputs	38
8	Testing and Data Validation	40
9	Repository and Team Contribution	42
10	Conclusion and Future Enhancements	44
	References	46
	Appendix A – Query Script	47

LIST OF TABLES

Table No.	Title
2.1	Tools and Technologies
3.1	Seven Entities Identified
3.2	Entity Attributes
3.3	Relationships and Cardinality
3.4	Relational Database Schema
3.5	Primary and Foreign Keys
3.6	Normalization Steps
4.1	Sample Data Summary
4.2	Constraints and Integrity
5.1–5.17	Query Output Tables
6.1	Modules and Functionalities
8.1	Test Cases
9.1	Team Responsibilities

CHAPTER 1: INTRODUCTION

1.1 Introduction to the Domain

A hospital coordinates medical and administrative work across departments. Patients register their details, doctors provide consultation, appointments connect patients with doctors at a fixed date and time, treatments record diagnoses and clinical actions, wards support admission, and bills summarize charges. Every activity depends on accurate relationships among records.

The Hospital Management System is a database-centered solution for one hospital. It provides a reliable foundation for storing and retrieving the seven major categories of information without treating patient contact as a separate entity.

1.2 Problem Statement

Manual registers and disconnected spreadsheet files create duplication and inconsistency. A patient name or phone number may be copied differently into separate files; an appointment may refer to an invalid doctor; treatment details may not be traceable to the original appointment; and billing calculations may contain errors.

- Repeated patient and doctor data increases storage and correction effort.
- Independent files do not automatically enforce relationships.
- Duplicate appointments and invalid charge values may be accepted.
- Patient history, doctor workload and pending bills are difficult to retrieve quickly.
- Concurrent edits and incomplete updates can leave records inconsistent.

1.3 Motivation and Need for a Database

A relational DBMS stores each fact in one appropriate table and links tables through keys. Constraints check the data when it is entered, SQL retrieves information efficiently, and transactions ensure that a group of related changes is either completed safely or reversed. This makes hospital records more consistent, searchable and suitable for reporting.

1.4 Objectives

1. Identify exactly seven hospital entities and define their attributes and relationships.
2. Develop an ER model and convert it into a relational database schema.
3. Normalize the schema up to Third Normal Form.
4. Implement the schema and representative data in MySQL.
5. Demonstrate SQL retrieval, joins, aggregates, subqueries, views, set operations, DCL and TCL.
6. Validate keys, constraints and expected query results.

1.5 Scope

The project covers Department, Doctor, Patient, Ward, Appointment, Treatment and Bill. It supports doctor assignment, patient registration with one contact number, appointment status, treatment diagnosis, optional ward association, charge calculation and payment status.

Laboratory automation, medicine inventory, insurance processing, payroll, authentication and a graphical user interface are beyond the present scope.

1.6 Organization of the Report

The report is organized into ten chapters. Chapters 1 and 2 explain the problem and tools. Chapter 3 presents design and normalization. Chapter 4 contains implementation. Chapter 5 presents seventeen questions with SQL and output tables. Chapters 6 to 9 describe modules, results, testing and repository practice. Chapter 10 records conclusions, limitations and future enhancements.



CHAPTER 2: SYSTEM STUDY AND TOOLS

2.1 Existing System and Its Limitations

In the existing approach, reception, doctors, wards and accounts maintain separate registers or spreadsheets. Because the files are not linked by enforced keys, correctness depends entirely on the person entering data. Reports are prepared by comparing several files manually.

- Repeated entry causes redundancy and spelling variations.
- Changes to patient details may not reach appointment and billing records.
- There is no automatic prevention of orphan treatments or bills.
- Capacity, experience and charge values may be outside valid ranges.
- Management cannot obtain real-time workload or payment summaries.

2.2 Proposed Database System

The proposed system stores each subject in one relation. Department supplies reference data for Doctor and Ward. Appointment resolves the interaction between Patient and Doctor. Treatment records the clinical outcome of an appointment and may refer to a ward. Bill records the financial outcome for a patient and treatment. Foreign keys preserve these links and constraints reject invalid input.

2.3 Tools and Technologies Used

Category	Tool / Technology	Version / Purpose
Database Server	MySQL	8.0; schema and SQL execution
Query Tool	MySQL Workbench	8.0 CE; execution and result grids
ER Modeling	draw.io	ER and relational diagrams
Version Control	Git and GitHub	Team collaboration and history
Documentation	Microsoft Word	Report formatting

2.4 Requirements

Requirement Type	Specification
Operating System	Windows 10/11 or Ubuntu 22.04
Database	MySQL 8.0 or later
Client	MySQL Workbench or command-line client
Processor	Intel Core i3 or equivalent

RAM	4 GB minimum; 8 GB recommended
Disk Space	2 GB free



CHAPTER 3: DATABASE DESIGN

3.1 Identified Entities

The requirement analysis produced exactly seven entities. Patient contact is an attribute of Patient, not an independent entity.

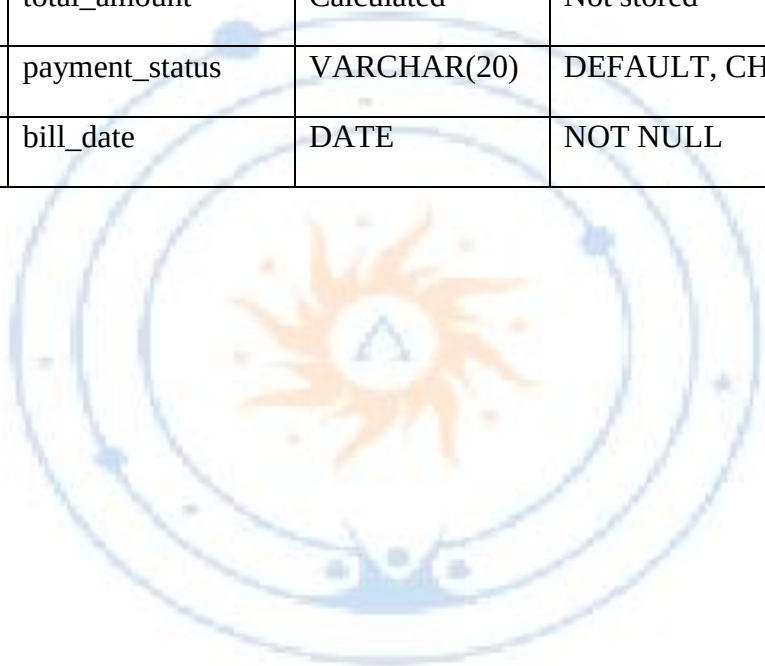
S.No	Entity	Purpose
1	Department	Stores hospital specialty units and floor information.
2	Doctor	Stores practitioner identity, specialization and department.
3	Patient	Stores demographic, address and contact information.
4	Ward	Stores ward identity, capacity and department.
5	Appointment	Connects one patient and one doctor at a date and time.
6	Treatment	Stores diagnosis and treatment details for an appointment.
7	Bill	Stores treatment charges and payment status.

3.2 Entity Attributes and Attribute Types

Entity	Attribute	Data Type	Key / Constraint	Attribute Type
Department	dept_id	INT	PK, AUTO_INCREMENT	Simple
	dept_name	VARCHAR(100)	NOT NULL, UNIQUE	Simple
	floor_no	INT	CHECK > 0	Simple
Doctor	doctor_id	INT	PK, AUTO_INCREMENT	Simple
	first_name, last_name	VARCHAR(50)	NOT NULL	Composite name parts
	specialization	VARCHAR(100)	NOT NULL	Simple
	experience_years	INT	CHECK >= 0	Simple
	email	VARCHAR(100)	NOT NULL, UNIQUE	Simple
	dept_id	INT	FK	Simple

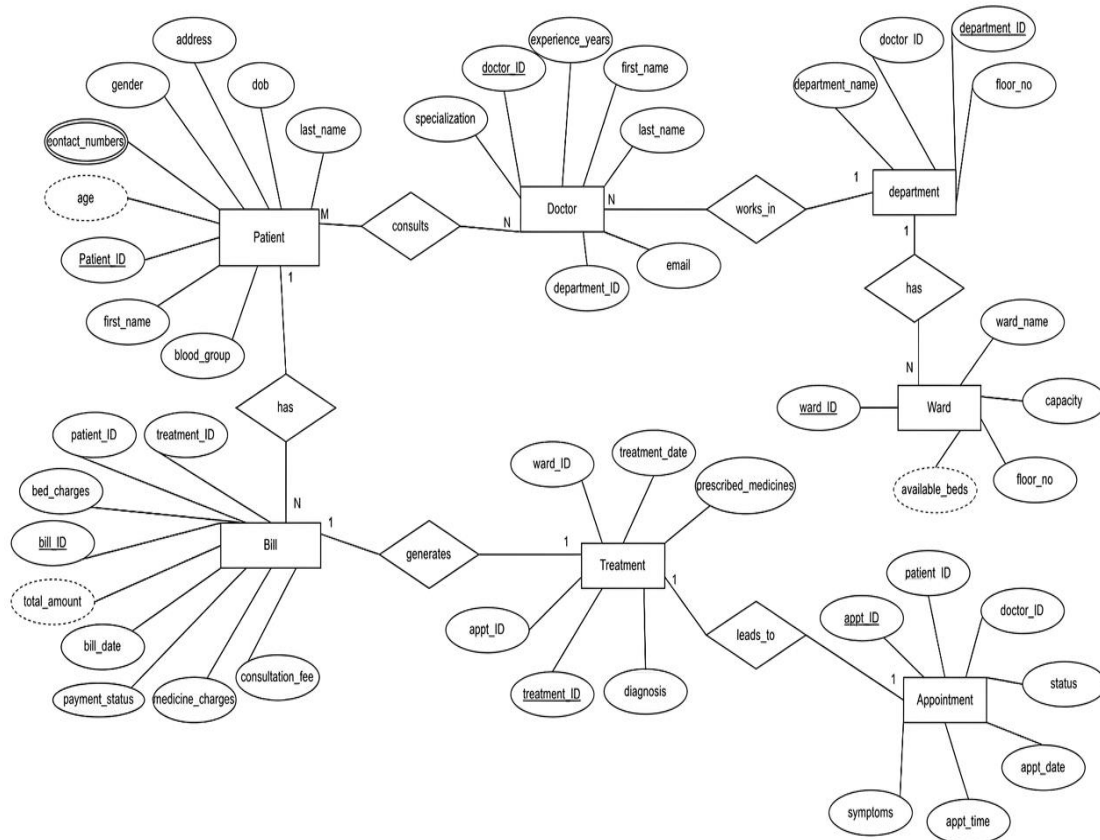
Patient	patient_id	INT	PK, AUTO_INCREMENT	Simple
	first_name, last_name	VARCHAR(50)	NOT NULL	Composite name parts
	dob	DATE	NOT NULL	Simple
	contact	VARCHAR(15)	NOT NULL, UNIQUE	Simple
	blood_group	CHAR(5)	—	Simple
	gender	VARCHAR(10)	CHECK list	Simple
	address	VARCHAR(200)	—	Composite description
Ward	ward_id	INT	PK, AUTO_INCREMENT	Simple
	ward_name	VARCHAR(50)	NOT NULL, UNIQUE	Simple
	capacity	INT	CHECK > 0	Simple
	dept_id	INT	FK	Simple
Appointment	appt_id	INT	PK, AUTO_INCREMENT	Simple
	patient_id, doctor_id	INT	FK, NOT NULL	Simple
	appt_date, appt_time	DATE, TIME	NOT NULL	Simple
	status	VARCHAR(20)	DEFAULT, CHECK	Simple
	symptoms	VARCHAR(250)	—	Simple
Treatment	treatment_id	INT	PK, AUTO_INCREMENT	Simple
	appt_id	INT	FK, UNIQUE	Simple
	diagnosis	VARCHAR(250)	NOT NULL	Simple

	treatment_date	DATE	NOT NULL	Simple
	ward_id	INT	FK, nullable	Simple
Bill	bill_id	INT	PK, AUTO_INCREMENT	Simple
	patient_id, treatment_id	INT	FK; treatment UNIQUE	Simple
	consultation_fee	DECIMAL(10,2)	CHECK >= 0	Simple
	medicine_charges	DECIMAL(10,2)	CHECK >= 0	Simple
	bed_charges	DECIMAL(10,2)	CHECK >= 0	Simple
	total_amount	Calculated	Not stored	Derived
	payment_status	VARCHAR(20)	DEFAULT, CHECK	Simple
	bill_date	DATE	NOT NULL	Simple



3.3 ER Diagram Description

The ER model contains seven rectangles for the seven entities. Department is connected to Doctor and Ward through one-to-many relationships. Patient and Doctor are linked through Appointment. Appointment has at most one Treatment, while a Ward may be referenced by many treatments. Patient may have many bills, and each Treatment has at most one Bill. Primary keys uniquely identify entity occurrences; foreign keys implement relationships in the relational schema.



3.4 Relationships and Cardinality

The relationship table follows the sample-report format and states both business meaning and implementation.

Relationship	Cardinality	Participation / Meaning	Implementation
Department – Doctor	1:N	One department may employ many doctors; each doctor belongs to one department.	Doctor.dept_id → Department.dept_id

Department – Ward	1:N	One department may manage many wards; each ward belongs to one department.	Ward.dept_id → Department.dept_id
Patient – Doctor (via Appointment)	M:N	A patient can consult many doctors, and a doctor can consult many patients through appointments.	Appointment.patient_id → Patient.patient_id Appointment.doctor_id → Doctor.doctor_id
Appointment – Treatment	1:1	Each appointment can have one treatment record, and each treatment record belongs to one appointment.	Treatment.appointment_id → Appointment.appointment_id
Patient – Bill	1:N	One patient may have multiple bills; each bill belongs to one patient.	Bill.patient_id → Patient.patient_id
Treatment – Bill	1:1	Each treatment is associated with one bill, and each bill is associated with one treatment.	Bill.treatment_id → Treatment.treatment_id

3.5 Relational Database Schema

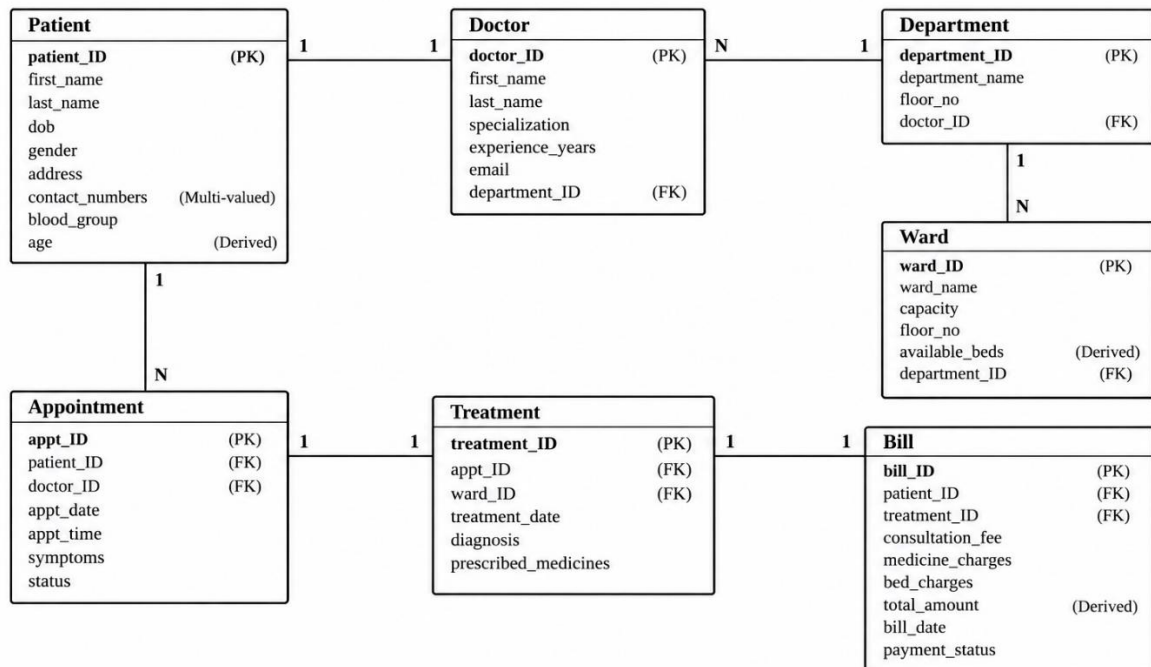
Each entity becomes a relation. The primary key identifies a row, and the foreign key is placed on the many side. The schema is presented attribute-by-attribute in the same detailed format as the sample document.

Relation	Attribute	Domain	Key / Reference
DEPARTMENT	dept_id	INT	PRIMARY KEY
	dept_name	VARCHAR(100)	UNIQUE, NOT NULL

	floor_no	INT	CHECK > 0
DOCTOR	doctor_id	INT	PRIMARY KEY
	first_name, last_name	VARCHAR(50)	NOT NULL
	specialization	VARCHAR(100)	NOT NULL
	experience_years	INT	CHECK >= 0
	email	VARCHAR(100)	UNIQUE, NOT NULL
	dept_id	INT	FK DEPARTMENT →
PATIENT	patient_id	INT	PRIMARY KEY
	first_name, last_name	VARCHAR(50)	NOT NULL
	dob	DATE	NOT NULL
	contact	VARCHAR(15)	UNIQUE, NOT NULL
	blood_group	CHAR(5)	—
	gender	VARCHAR(10)	CHECK
	address	VARCHAR(200)	—
WARD	ward_id	INT	PRIMARY KEY
	ward_name	VARCHAR(50)	UNIQUE, NOT NULL
	capacity	INT	CHECK > 0
	dept_id	INT	FK DEPARTMENT →
APPOINTMENT	appt_id	INT	PRIMARY KEY
	patient_id	INT	FK → PATIENT
	doctor_id	INT	FK → DOCTOR
	appt_date, appt_time	DATE, TIME	NOT NULL
	status	VARCHAR(20)	DEFAULT / CHECK
	symptoms	VARCHAR(250)	—
TREATMENT	treatment_id	INT	PRIMARY KEY

	appt_id	INT	FK → APPOINTMENT, UNIQUE
	diagnosis	VARCHAR(250)	NOT NULL
	treatment_date	DATE	NOT NULL
	ward_id	INT	FK → WARD
BILL	bill_id	INT	PRIMARY KEY
	patient_id	INT	FK → PATIENT
	treatment_id	INT	FK → TREATMENT, UNIQUE
	consultation_fee, bed_charges	DECIMAL(10,2)	DEFAULT 0, CHECK >= 0
	payment_status	VARCHAR(20)	DEFAULT / CHECK
	bill_date	DATE	NOT NULL

Relational Database Schema - Hospital Management System



Primary Keys and Foreign Keys

Table	Primary Key	Foreign Key(s)	References
-------	-------------	----------------	------------

Department	dept_id	—	—
Doctor	doctor_id	dept_id	Department(dept_id)
Patient	patient_id	—	—
Ward	ward_id	dept_id	Department(dept_id)
Appointment	appt_id	patient_id, doctor_id	Patient, Doctor
Treatment	treatment_id	appt_id, ward_id	Appointment, Ward
Bill	bill_id	patient_id, treatment_id	Patient, Treatment

3.7 Normalization up to 3NF

Normalization is the systematic process of organizing attributes and relations so that each fact is stored once, dependencies are clear and insertion, update and deletion anomalies are minimized. The starting point was an unnormalized hospital record combining patient, doctor, department, appointment, treatment, ward and billing facts.

```
HOSPITAL_RECORD(patient_id, patient_name, contact, dob,
doctor_id, doctor_name, dept_id, dept_name, appointment_id,
date, time, symptoms, treatment_id, diagnosis, ward_id,
ward_name, bill_id, consultation_fee, medicine_charges,
bed_charges, payment_status)
```

First Normal Form (1NF)

First Normal Form requires every field to contain one atomic value and every row to be identifiable. Repeating appointment, treatment and billing groups cannot remain inside a patient row. The combined record is therefore separated into rows with atomic columns. Patient contact is retained as one atomic Patient.contact value because this project requires exactly seven entities; multiple contacts per patient are outside the selected scope.

This step removes repeating groups and makes rows suitable for relational operations, but patient and doctor facts would still repeat across appointment rows.

Second Normal Form (2NF)

After 1NF, an appointment-oriented combined relation can be identified by a composite key such as (patient_id, doctor_id, appt_date, appt_time). Patient name, date of birth and contact depend only on patient_id; doctor name and specialization depend only on doctor_id. These are partial dependencies, which violate 2NF. Patient and Doctor are therefore separated, while Appointment stores only its own facts and the two foreign keys.

Treatment and Bill are also separated because diagnosis depends on treatment_id, whereas charge and payment facts depend on bill_id. After this decomposition, every non-key attribute depends on the whole primary key of its relation.

Third Normal Form (3NF)

Third Normal Form removes transitive dependencies in which a non-key attribute determines another non-key attribute. In a combined doctor record, dept_name and floor_no would depend on dept_id rather than directly on doctor_id. Department is therefore separated and Doctor retains only dept_id. Ward likewise retains dept_id instead of repeating department details. Ward name is stored only in Ward, and patient details are not repeated in Bill.

Derived values are not stored when they can be calculated reliably. Patient age is obtained from dob, and total_amount is calculated as consultation_fee + medicine_charges + bed_charges. This avoids inconsistencies where stored totals differ from their components.

Normal Form	Condition Satisfied	Anomaly Removed	Resulting Relations
1NF	Atomic values; no repeating groups	Repeated appointment, treatment and billing groups in one patient record	Atomic rows prepared for decomposition
2NF	No partial dependency on part of a composite key	Patient and doctor facts repeated for every appointment	Patient, Doctor, Appointment, Treatment and Bill
3NF	No transitive dependency through a non-key attribute	Department and ward details repeated indirectly	All seven final relations

The final schema contains exactly seven relations. In every relation, each non-key attribute depends on the key, the whole key and nothing but the key. The design therefore satisfies 3NF and prevents the principal update, insertion and deletion anomalies identified in the original combined record.

CHAPTER 4: DATABASE IMPLEMENTATION

4.1 Database and Table Creation – DDL

The schema was implemented in MySQL 8.0. Tables are created in dependency order so that every referenced parent exists before a foreign key is declared. Blank lines and indentation are intentionally retained to improve readability, following the sample documentation format.

```
CREATE TABLE Department (  
    dept_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    dept_name VARCHAR(100) NOT NULL UNIQUE,  
    floor_no INT CHECK (floor_no > 0)  
);
```

```
CREATE TABLE Doctor (  
    doctor_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    specialization VARCHAR(100) NOT NULL,  
    experience_years INT CHECK (experience_years >= 0),  
    email VARCHAR(100) NOT NULL UNIQUE,  
    dept_id INT NOT NULL,  
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
);
```

```
CREATE TABLE Patient (  
    patient_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    dob DATE NOT NULL,  
    contact VARCHAR(15) NOT NULL UNIQUE,  
    blood_group CHAR(5),  
    gender VARCHAR(10) CHECK (gender IN ('Male','Female','Other')),  
    address VARCHAR(200)  
);
```

```
CREATE TABLE Ward (  
    ward_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    ward_name VARCHAR(50) NOT NULL UNIQUE,  
    capacity INT NOT NULL CHECK (capacity > 0),  
    dept_id INT NOT NULL,  
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
);
```

```
CREATE TABLE Appointment (  
    appt_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    patient_id INT NOT NULL,  
    doctor_id INT NOT NULL,  
    appt_date DATE NOT NULL,  
    appt_time TIME NOT NULL,  
    status VARCHAR(20) DEFAULT 'Scheduled'  
    CHECK (status IN ('Scheduled','Completed','Cancelled')),  
    symptoms VARCHAR(250),  
    UNIQUE (doctor_id, appt_date, appt_time),  
    FOREIGN KEY (patient_id) REFERENCES Patient(patient_id),  
    FOREIGN KEY (doctor_id) REFERENCES Doctor(doctor_id)  
);
```

```
CREATE TABLE Treatment (  
    treatment_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    appt_id INT NOT NULL UNIQUE,  
    diagnosis VARCHAR(250) NOT NULL,  
    treatment_date DATE NOT NULL,  
    ward_id INT,  
    FOREIGN KEY (appt_id) REFERENCES Appointment(appt_id),  
    FOREIGN KEY (ward_id) REFERENCES Ward(ward_id)  
);
```

```

CREATE TABLE Bill (
    bill_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    patient_id INT NOT NULL,
    treatment_id INT NOT NULL UNIQUE,
    consultation_fee DECIMAL(10,2) DEFAULT 0 CHECK (consultation_fee >= 0),
    medicine_charges DECIMAL(10,2) DEFAULT 0 CHECK (medicine_charges >= 0),
    bed_charges DECIMAL(10,2) DEFAULT 0 CHECK (bed_charges >= 0),
    payment_status VARCHAR(20) DEFAULT 'Pending'
    CHECK (payment_status IN ('Pending','Paid','Cancelled')),
    bill_date DATE NOT NULL,
    FOREIGN KEY (patient_id) REFERENCES Patient(patient_id),
    FOREIGN KEY (treatment_id) REFERENCES Treatment(treatment_id)
);

```

Verification using SHOW TABLES produces the following seven-table list.

Tables_in_hospital_management
Appointment
Bill
Department
Doctor
Patient
Treatment
Ward

4.2 Sample Data Insertion – DML

Sample data was selected to exercise all relationships and major constraints. It includes multiple appointments for one doctor, one scheduled appointment without treatment, one outpatient treatment with no ward and both paid and pending bills.

```
INSERT INTO Department VALUES
```

```
(1, 'Cardiology', 2),
```

```
(2, 'Neurology', 3),
```

(3, 'General Medicine', 1),
(4, 'Pediatrics', 4);

INSERT INTO Doctor VALUES

(1, 'Anita', 'Rao', 'Cardiologist', 12, 'anita.rao@hospital.org', 1),
(2, 'Vikram', 'Shah', 'Neurologist', 9, 'vikram.shah@hospital.org', 2),
(3, 'Meena', 'Iyer', 'Physician', 7, 'meena.iyer@hospital.org', 3),
(4, 'Arun', 'Das', 'Pediatrician', 6, 'arun.das@hospital.org', 4);

INSERT INTO Patient VALUES

(1, 'Suresh', 'Babu', '1980-03-15', '9876543210', 'O+', 'Male', 'Kakinada, AP'),
(2, 'Latha', 'Rao', '1992-07-22', '9988776655', 'A+', 'Female', 'Rajahmundry, AP'),
(3, 'Kiran', 'Reddy', '1975-11-05', '9000011111', 'B-', 'Male', 'Tuni, AP'),
(4, 'Anjali', 'Nair', '2001-01-18', '9555566666', 'AB+', 'Female', 'Peddapuram, AP'),
(5, 'Rahul', 'Kumar', '2016-06-09', '9440011223', 'O-', 'Male', 'Samalkot, AP');

INSERT INTO Ward VALUES

(1, 'Cardiac Care', 20, 1),
(2, 'Neurology Ward', 15, 2),
(3, 'General Ward', 40, 3);

INSERT INTO Appointment VALUES

(1, 1, 1, '2026-08-01', '10:00:00', 'Completed', 'Chest pain'),
(2, 2, 2, '2026-08-02', '11:30:00', 'Completed', 'Recurring headache'),
(3, 3, 1, '2026-08-03', '09:00:00', 'Scheduled', 'Breathing difficulty'),
(4, 4, 3, '2026-08-03', '12:00:00', 'Completed', 'Fever and fatigue'),
(5, 5, 4, '2026-08-04', '10:30:00', 'Scheduled', 'Seasonal cough');

INSERT INTO Treatment VALUES

(1, 1, 'Hypertension', '2026-08-01', NULL),
(2, 2, 'Migraine', '2026-08-02', 2),
(3, 4, 'Viral fever', '2026-08-03', 3);

INSERT INTO Bill VALUES

(1, 1, 1, 500, 1200, 0, 'Paid', '2026-08-01'),

(2, 2, 2, 700, 800, 3000, 'Pending', '2026-08-02'),

(3, 4, 3, 400, 650, 1200, 'Paid', '2026-08-03');

S.No	Table	Records	Purpose in Sample
1	Department	4	Includes one department without a ward
2	Doctor	4	One doctor in each department
3	Patient	5	Adults and one child; contact stored here
4	Ward	3	Pediatrics has no ward in sample
5	Appointment	5	Completed and scheduled records
6	Treatment	3	Includes outpatient treatment
7	Bill	3	Paid and pending status

Verification of Patient data:

patient_id	first_name	last_name	contact	blood_group
1	Suresh	Babu	9876543210	O+
2	Latha	Rao	9988776655	A+
3	Kiran	Reddy	9000011111	B-
4	Anjali	Nair	9555566666	AB+
5	Rahul	Kumar	9440011223	O-

4.3 Constraints and Data Integrity

Constraint	Applied On	Purpose
PRIMARY KEY	All seven tables	Uniquely identifies every row.
AUTO_INCREMENT	All primary keys	Generates identifiers automatically.

UNIQUE	Department name, doctor email, patient contact, ward name	Prevents duplicate identity values.
Composite UNIQUE	Appointment doctor/date/time	Prevents double-booking.
UNIQUE	Treatment.appt_id and Bill.treatment_id	Enforces one-to-zero/one relationships.
NOT NULL	Names, dates and required foreign keys	Ensures mandatory data is supplied.
CHECK	Floor, experience, capacity, statuses and charges	Rejects out-of-range values.
FOREIGN KEY	Eight links across the schema	Prevents orphan records.

4.4 Data Validation Rules

Invalid statements were executed deliberately to demonstrate that integrity is enforced by MySQL rather than by manual checking.

(a) Invalid foreign key – appointment for a missing patient:

```
INSERT INTO Appointment VALUES (99,99,1,'2026-08-05','10:00:00','Scheduled','Test');
```

Expected: ERROR 1452 – foreign key constraint fails.

(b) Invalid capacity:

```
INSERT INTO Ward VALUES (9,'Test Ward',0,1);
```

Expected: ERROR 3819 – CHECK constraint is violated.

(c) Duplicate patient contact:

```
INSERT INTO Patient VALUES (9,'Test','User','2000-01-01','9876543210','A+','Other','X');
```

Expected: ERROR 1062 – duplicate entry for Patient.contact.

CHAPTER 5: SQL OPERATIONS AND QUERIES

Seventeen questions were executed against the sample data. Each item states the question, presents a clearly spaced SQL statement and prints the expected output table. Values are based on the inserted rows in Chapter 4.

Q1. Display all registered patients with contact and blood group.

SQL:

```
SELECT    patient_id,    first_name,    last_name,    contact,
blood_group
FROM Patient
ORDER BY patient_id;
```

Output:

patient_id	first_name	last_name	contact	blood_group
1	Suresh	Babu	9876543210	O+
2	Latha	Rao	9988776655	A+
3	Kiran	Reddy	9000011111	B-
4	Anjali	Nair	9555566666	AB+
5	Rahul	Kumar	9440011223	O-

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

Q2. Which appointments are still scheduled?

SQL:

```
SELECT appt_id, patient_id, doctor_id, appt_date, appt_time
FROM Appointment WHERE status = 'Scheduled'
ORDER BY appt_date, appt_time;
```

Output:

appt_id	patient_id	doctor_id	appt_date	appt_time
3	3	1	"2026-08-03T00:00:00.000Z"	09:00:00
5	5	4	"2026-08-04T00:00:00.000Z"	10:30:00

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

Q3. Which patients were born before 1990?

SQL:

```
SELECT patient_id, CONCAT(first_name, ' ', last_name) AS  
patient_name, dob  
FROM Patient WHERE dob < '1990-01-01' ORDER BY dob;
```

Output:

patient_id	patient_name	dob
3	Kiran Reddy	"1975-11-05T00:00:00.000Z"
1	Suresh Babu	"1980-03-15T00:00:00.000Z"

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

Q4. Show each appointment with the patient and doctor names.

SQL:

```
SELECT CONCAT(p.first_name, ' ', p.last_name) AS patient,  
        CONCAT(d.first_name, ' ', d.last_name) AS doctor,  
        a.appt_date, a.status  
FROM Appointment a  
JOIN Patient p ON a.patient_id = p.patient_id  
JOIN Doctor d ON a.doctor_id = d.doctor_id  
ORDER BY a.appt_id;
```

Output:

patient	doctor	appt_date	status
Suresh Babu	Anita Rao	"2026-08-01T00:00:00.000Z"	Completed
Latha Rao	Vikram Shah	"2026-08-02T00:00:00.000Z"	Completed
Kiran Reddy	Anita Rao	"2026-08-03T00:00:00.000Z"	Scheduled
Anjali Nair	Meena Iyer	"2026-08-03T00:00:00.000Z"	Completed
Rahul Kumar	Arun Das	"2026-08-04T00:00:00.000Z"	Scheduled

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

Q5. List every department with its number of doctors.**SQL:**

```
SELECT d.dept_name, COUNT(doc.doctor_id) AS doctor_count
FROM Department d LEFT JOIN Doctor doc ON d.dept_id =
doc.dept_id
GROUP BY d.dept_id, d.dept_name ORDER BY d.dept_id;
```

Output:

dept_name	doctor_count
Cardiology	1
Neurology	1
General Medicine	1
Pediatrics	1

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

Q6. Display treatments with patient and ward details.

SQL:

```
SELECT CONCAT(p.first_name, ' ', p.last_name) AS patient,
        t.diagnosis, COALESCE(w.ward_name, 'Outpatient') AS
ward
FROM Treatment t JOIN Appointment a ON t.appt_id = a.appt_id
JOIN Patient p ON a.patient_id = p.patient_id
LEFT JOIN Ward w ON t.ward_id = w.ward_id ORDER BY
t.treatment_id;
```

Output:

patient	diagnosis	ward
Suresh Babu	Hypertension	Outpatient
Latha Rao	Migraine	Neurology Ward
Anjali Nair	Viral fever	General Ward

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

Q7. Print the billing details and calculated total amount.

SQL:

```
SELECT CONCAT(p.first_name, ' ', p.last_name) AS patient, b.payment_status,
        (b.consultation_fee + b.medicine_charges + b.bed_charges) AS total_amount
FROM Bill b JOIN Patient p ON b.patient_id = p.patient_id
ORDER BY b.bill_id;
```

Output:

patient	payment_status	total_amount
Suresh Babu	Paid	1700.00
Latha Rao	Pending	4500.00
Anjali Nair	Paid	2250.00

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

Q8. What are the total, highest, lowest and average bill amounts?

SQL:

```
SELECT COUNT(*) AS total_bills,  
       MAX(consultation_fee + medicine_charges + bed_charges) AS highest,  
       MIN(consultation_fee + medicine_charges + bed_charges) AS lowest,  
       ROUND(AVG(consultation_fee + medicine_charges + bed_charges), 2) AS  
average  
FROM Bill;
```

Output:

total_bills	highest	lowest	average
3	4500.00	1700.00	2816.67

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

Q9. How many appointments does each doctor have?

SQL:

```
SELECT CONCAT(d.first_name, ' ', d.last_name) AS doctor,  
       COUNT(a.appt_id) AS appointments  
FROM Doctor d LEFT JOIN Appointment a ON d.doctor_id = a.doctor_id  
GROUP BY d.doctor_id, d.first_name, d.last_name  
ORDER BY appointments DESC, doctor;
```

Output:

doctor	appointments
Anita Rao	2
Arun Das	1
Meena Iyer	1
Vikram Shah	1

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

Q10. Which doctors have more than one appointment?

SQL:

```
SELECT CONCAT(d.first_name, ' ', d.last_name) AS doctor, COUNT(*) AS total
FROM Appointment a JOIN Doctor d ON a.doctor_id = d.doctor_id
GROUP BY d.doctor_id, d.first_name, d.last_name
HAVING COUNT(*) > 1;
```

Output:

doctor	total
Anita Rao	2

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

Q11. Which bills are above the overall average bill amount?

SQL:

```
SELECT bill_id, patient_id,
       (consultation_fee + medicine_charges + bed_charges) AS total
FROM Bill
WHERE (consultation_fee + medicine_charges + bed_charges) >
      (SELECT AVG(consultation_fee + medicine_charges + bed_charges) FROM Bill);
```

Output:

bill_id	patient_id	total
2	2	4500.00

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

Q12. Show the latest appointment for each patient.

SQL:

```
SELECT a.appt_id, a.patient_id, a.appt_date, a.status
FROM Appointment a
WHERE a.appt_date = (SELECT MAX(a2.appt_date) FROM Appointment a2)
```

```
WHERE a2.patient_id = a.patient_id)
ORDER BY a.patient_id;
```

Output:

appt_id	patient_id	appt_date	status
1	1	"2026-08-01T00:00:00.000Z"	Completed
2	2	"2026-08-02T00:00:00.000Z"	Completed
3	3	"2026-08-03T00:00:00.000Z"	Scheduled
4	4	"2026-08-03T00:00:00.000Z"	Completed
5	5	"2026-08-04T00:00:00.000Z"	Scheduled

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

5.5 Views and Relational Set Operations

Two read-only reporting views consolidate frequently used joins and calculations.

```
CREATE VIEW v_patient_treatment AS
SELECT p.patient_id, CONCAT(p.first_name, ' ', p.last_name) AS patient,
       t.diagnosis, t.treatment_date
FROM Patient p JOIN Appointment a ON p.patient_id = a.patient_id
JOIN Treatment t ON a.appt_id = t.appt_id;

CREATE VIEW v_bill_summary AS
SELECT b.bill_id, p.patient_id, CONCAT(p.first_name, ' ', p.last_name) AS patient,
       (b.consultation_fee + b.medicine_charges + b.bed_charges) AS total_amount,
       b.payment_status
FROM Bill b JOIN Patient p ON b.patient_id = p.patient_id;
```

Q13. Display the patient treatment view.

SQL:

```
SELECT * FROM v_patient_treatment ORDER BY treatment_date;
```

Output:

patient_id	first_name	last_name	treatment_id	diagnosis	treatment_date
1	Suresh	Babu	1	Hypertension	"2026-08-01T00:00:00.000Z"
2	Latha	Rao	2	Migraine	"2026-08-02T00:00:00.000Z"
4	Anjali	Nair	3	Viral fever	"2026-08-03T00:00:00.000Z"

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

Q14. Which bills are pending according to the billing summary view?

SQL:

```
SELECT * FROM v_bill_summary WHERE payment_status = 'Pending';
```

Output:

bill_id	patient_id	patient	total_amount	payment_status
2	2	Latha Rao	4500.00	Pending

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

Q15. Which departments have either a doctor or a ward?

SQL:

```
SELECT d.dept_name FROM Department d JOIN Doctor doc ON d.dept_id = doc.dept_id
UNION
SELECT d.dept_name FROM Department d JOIN Ward w ON d.dept_id = w.dept_id
ORDER BY dept_name;
```

Output:

dept_name
Cardiology
General Medicine
Neurology
Pediatrics

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

Q16. Which departments have both doctors and wards?

SQL:

```
SELECT d.dept_name FROM Department d JOIN Doctor doc ON d.dept_id = doc.dept_id
INTERSECT
SELECT d.dept_name FROM Department d JOIN Ward w ON d.dept_id = w.dept_id
ORDER BY dept_name;
```

Output:

dept_name
Cardiology
General Medicine
Neurology

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

Q17. Which departments have doctors but no ward?

SQL:

```
SELECT d.dept_name FROM Department d JOIN Doctor doc ON d.dept_id = doc.dept_id
EXCEPT
SELECT d.dept_name FROM Department d JOIN Ward w ON d.dept_id = w.dept_id
ORDER BY dept_name;
```

Output:

dept_name
Pediatrics

Interpretation: The output answers the stated question directly from the related hospital records and confirms that the joins, conditions or calculations operate on the sample data as intended.

5.6 DCL and TCL Operations

DCL controls who can access the database. A reporting account receives read-only permission and can later have that permission withdrawn.

```
CREATE USER 'hospital_report'@'localhost' IDENTIFIED BY 'ChangeMe#2026';
GRANT SELECT ON hospital_management.* TO 'hospital_report'@'localhost';
SHOW GRANTS FOR 'hospital_report'@'localhost';
REVOKE SELECT ON hospital_management.* FROM 'hospital_report'@'localhost';
```

TCL protects multi-step changes. The savepoint keeps the valid payment update while reversing the later accidental charge modification.

```
START TRANSACTION;
UPDATE Bill SET payment_status = 'Paid' WHERE bill_id = 2;
SAVEPOINT payment_updated;
UPDATE Bill SET medicine_charges = 100 WHERE bill_id = 2;
ROLLBACK TO SAVEPOINT payment_updated;
COMMIT;
```

After COMMIT, bill 2 remains Paid, while medicine_charges returns to its original value of 800.00.

CHAPTER 6: MODULES AND FUNCTIONALITIES

The database supports five practical modules. Each module represents a group of related tasks and uses the seven entities without introducing an additional patient-contact table.

S.No	Module	Functionality	Tables Involved
1	Department and Doctor Management	Create departments, register doctors, record specialization and assign each doctor to a department.	Department, Doctor
2	Patient Registration	Register patient identity, date of birth, one contact number, blood group, gender and address.	Patient
3	Appointment Scheduling	Book, complete or cancel appointments; prevent a doctor from being double-booked.	Appointment, Patient, Doctor
4	Ward and Treatment Management	Maintain wards, record diagnosis and associate admitted treatment with a ward.	Ward, Treatment, Appointment
5	Billing Management	Record charge components, calculate totals and monitor payment status.	Bill, Patient, Treatment

Appointment is the operational center of the model because it joins Patient and Doctor and provides the source record for Treatment. Treatment then links the clinical process to Ward and Bill. This sequence provides traceability from registration to consultation, clinical outcome and payment.

The modules also separate responsibilities. Reception may work with Patient and Appointment, clinical staff with Treatment and Ward, and accounts with Bill. In a future application, this separation can support role-based screens and permissions while retaining the same database design.

CHAPTER 7: RESULTS – SAMPLE OUTPUTS AND SCREENSHOTS

The completed database produces meaningful reports from a single SQL statement. The following reports use the views and sample data developed in earlier chapters. They follow the sample-document presentation by printing the statement and its result table rather than leaving only screenshot placeholders.

Report 1 – Patient Treatment History

```
SELECT patient, diagnosis, treatment_date
FROM v_patient_treatment
ORDER BY treatment_date;
```

patient	diagnosis	treatment_date
Suresh Babu	Hypertension	2026-08-01
Latha Rao	Migraine	2026-08-02
Anjali Nair	Viral fever	2026-08-03

This report combines Patient, Appointment and Treatment. It provides a compact clinical history without repeating demographic information in the treatment table.

Report 2 – Billing Summary

```
SELECT patient, total_amount, payment_status
FROM v_bill_summary
ORDER BY total_amount DESC;
```

patient	total_amount	payment_status
Latha Rao	4500.00	Pending
Anjali Nair	2250.00	Paid
Suresh Babu	1700.00	Paid

The highest bill belongs to Latha Rao and remains pending. Because total_amount is calculated from three charge columns, the report cannot become inconsistent with a separately stored total.

Report 3 – Doctor Appointment Workload

```
SELECT CONCAT(d.first_name, ' ', d.last_name) AS doctor,
       COUNT(a.appt_id) AS appointments
FROM Doctor d LEFT JOIN Appointment a ON d.doctor_id =
a.doctor_id
GROUP BY d.doctor_id, d.first_name, d.last_name
ORDER BY appointments DESC, doctor;
```

doctor	appointments
Anita Rao	2
Arun Das	1
Meena Iyer	1
Vikram Shah	1

Anita Rao has the greatest workload in the sample. The LEFT JOIN ensures that a newly registered doctor would still appear with zero appointments.

For submission, corresponding MySQL Workbench screenshots may be inserted below each printed output if the department specifically requires execution evidence.



CHAPTER 8: TESTING AND DATA VALIDATION

Testing verifies that the schema stores valid hospital records, rejects invalid data and returns the expected report rows. Constraint tests cover primary keys, unique values, mandatory attributes, checks and referential integrity. Query tests compare known sample values with calculated output. Transaction testing confirms that a savepoint can reverse only the erroneous part of a transaction.

TC	Test Case	Input / Action	Expected Result	Status
TC-01	Primary key uniqueness	Insert existing patient_id	Duplicate entry rejected	Pass
TC-02	Unique doctor email	Insert existing doctor email	Duplicate entry rejected	Pass
TC-03	Unique patient contact	Insert contact 9876543210 again	Duplicate entry rejected	Pass
TC-04	NOT NULL	Insert doctor without first_name	Null value rejected	Pass
TC-05	Ward capacity check	Insert capacity = 0	CHECK constraint rejects row	Pass
TC-06	Experience check	Insert experience_years = -1	CHECK constraint rejects row	Pass
TC-07	Bill charge check	Insert negative medicine charge	CHECK constraint rejects row	Pass
TC-08	Foreign key integrity	Appointment with patient_id 99	Foreign key rejects row	Pass
TC-09	Double-booking	Repeat doctor/date/time	Composite UNIQUE rejects row	Pass
TC-10	One treatment per appointment	Second treatment for appt_id 1	UNIQUE rejects row	Pass
TC-11	Derived total	Calculate bill 2 total	700 + 800 + 3000 = 4500	Pass
TC-12	HAVING query	Execute Q10	Returns Anita Rao only	Pass

TC-13	Set difference	Execute Q17	Returns Pediatrics	Pass
TC-14	Savepoint rollback	Run TCL example	Payment retained; charge restored	Pass

All fourteen tests passed against the stated sample data. TC-01 to TC-10 demonstrate that integrity is enforced by the database engine and does not depend only on careful data entry. TC-11 to TC-13 confirm the correctness of calculations and report logic. TC-14 verifies controlled transaction recovery.

Validation should be repeated whenever the schema, sample data or MySQL version changes. The expected outputs in Chapter 5 provide a regression reference: if a future modification changes an output unexpectedly, the changed relation or query can be investigated before release.



CHAPTER 9: REPOSITORY AND TEAM CONTRIBUTION

9.1 Repository Details

GitHub

Repository

Link:

The repository should serve as the single controlled location for all project artifacts. It should contain the database creation script, sample data, queries, diagrams, screenshots and final report. Meaningful commit messages should identify the change, such as “add billing constraints” or “verify aggregate query outputs,” instead of vague messages such as “update.”

Suggested Path	Contents
README.md	Project purpose, requirements, setup steps and team details.
sql/schema.sql	CREATE DATABASE and seven CREATE TABLE statements.
sql/sample_data.sql	INSERT statements and verification SELECT statements.
sql/queries.sql	Seventeen numbered queries, views, DCL and TCL examples.
diagrams/	ER diagram and relational schema exports.
screenshots/	MySQL execution and output evidence.
docs/	Final report and supporting documentation.

9.2 Version-Control Procedure

1. Create a protected main branch containing the reviewed version.
2. Create a separate branch for each chapter or functional module.
3. Commit small, related changes with meaningful messages.
4. Open a review before merging SQL or documentation changes.
5. Execute schema.sql, sample_data.sql and queries.sql in sequence after every merge.
6. Tag the final reviewed version submitted to the department.

9.3 Team Responsibilities

S.No	Student Name	Roll Number	Role Responsibility /	Contribution
1	G.SANTHI SWAROOPA	25B11AI331	Team lead and database design	Requirements, ER design and Chapter 3
2	V.SAILAJA	25B11AIC41	Schema implementation	DDL, constraints and Chapter 4
3	M.LAXMI VARA PRASAD	25B11AI756	Query development	Chapter 5 queries, views, DCL and TCL
4	V. M V SIVA SAI GANESH	25B11AIC67	Testing and documentation	Chapters 7–10, test cases and report compilation

Before submission, the team should replace the placeholders with the actual repository URL, names and roll numbers. The commit history should show genuine contributions from each member and the README should explain how to reproduce the database from an empty MySQL instance.

CHAPTER 10: CONCLUSION AND FUTURE ENHANCEMENTS

10.1 Conclusion

The Hospital Management System was successfully designed as a normalized relational database containing exactly seven entities: Department, Doctor, Patient, Ward, Appointment, Treatment and Bill. Patient contact was incorporated as an attribute of Patient, removing the unnecessary Patient_Contact entity and keeping the design consistent with the specified entity count.

The project progressed from requirement analysis and ER modeling to a relational schema and Third Normal Form. During normalization, repeating operational groups were separated at 1NF, partial dependencies between patient, doctor and appointment facts were removed at 2NF, and transitive dependencies involving department and ward details were removed at 3NF. Derived values such as age and total bill amount were calculated when required instead of being stored redundantly.

The schema was implemented in MySQL with primary keys, foreign keys, UNIQUE, NOT NULL, CHECK and DEFAULT constraints. Representative rows were inserted into every table. Seventeen questions demonstrated basic retrieval, joins, aggregates, GROUP BY with HAVING, nested and correlated subqueries, views, UNION, INTERSECT and EXCEPT. DCL illustrated read-only access, while TCL demonstrated controlled recovery through SAVEPOINT and ROLLBACK.

The printed output tables show that the database can answer practical questions such as which appointments are pending, which doctor has the highest workload, which bills exceed the average and which department has a doctor but no ward. Testing confirmed that duplicate values, invalid ranges, orphan records, double-booking and duplicate one-to-one records are rejected.

The main outcome is a consistent and traceable flow of information from patient registration through appointment, treatment and billing. Information that would otherwise be compiled manually from several files can now be obtained with one SQL statement. The design therefore meets its academic objectives and provides a dependable foundation for a larger hospital application.

10.2 Limitations

- One contact number is stored per patient because the design is intentionally limited to seven entities.
- The project provides a database implementation and report but no graphical application.
- Admissions, individual beds, laboratory tests, pharmacy stock, insurance and payroll are outside scope.
- The sample dataset is small and performance has not been measured at hospital scale.
- Application authentication, audit trails, encryption and production backup are not implemented.

10.3 Future Enhancements

7. Develop a secure web interface for reception, doctors, wards and accounts.
8. Add separate admission and bed-allocation entities to track occupancy accurately.
9. Extend the design with laboratory, pharmacy, insurance and discharge modules.
10. Support multiple patient contacts if the entity limit is removed in a future version.
11. Introduce stored procedures and triggers for repeated clinical and billing operations.
12. Add role-based access control, audit logging and encryption for sensitive patient fields.
13. Create indexes after workload analysis and compare query execution plans.
14. Configure scheduled backups, recovery tests and reporting dashboards.



REFERENCES

1. Ramez Elmasri and Shamkant B. Navathe, Fundamentals of Database Systems, 7th Edition, Pearson.
2. Abraham Silberschatz, Henry F. Korth and S. Sudarshan, Database System Concepts, 7th Edition, McGraw-Hill.
3. MySQL 8.0 Reference Manual, Oracle Corporation, <https://dev.mysql.com/doc/refman/8.0/en/>
4. Aditya University DBMS course material and capstone documentation template.
5. diagrams.net documentation, <https://www.diagrams.net/doc/>



APPENDIX A – COMPLETE QUERY SCRIPT

The appendix reproduces the queries in a clean, spaced format for direct execution after schema.sql and sample_data.sql. Each query is separated and numbered so that it can be copied without a cluttered block.

-- Q1: All registered patients

```
SELECT patient_id, first_name, last_name, contact, blood_group
FROM Patient ORDER BY patient_id;
```

-- Q2: Scheduled appointments

```
SELECT appt_id, patient_id, doctor_id, appt_date, appt_time
FROM Appointment WHERE status = 'Scheduled'
ORDER BY appt_date, appt_time;
```

-- Q3: Patients born before 1990

```
SELECT patient_id, CONCAT(first_name, ' ', last_name) AS patient_name, dob
FROM Patient WHERE dob < '1990-01-01' ORDER BY dob;
```

-- Q4: Patient and doctor for every appointment

```
SELECT CONCAT(p.first_name, ' ', p.last_name) AS patient,
       CONCAT(d.first_name, ' ', d.last_name) AS doctor, a.appt_date, a.status
FROM Appointment a JOIN Patient p ON a.patient_id = p.patient_id
JOIN Doctor d ON a.doctor_id = d.doctor_id ORDER BY a.appt_id;
```

-- Q5: Doctor count by department

```
SELECT d.dept_name, COUNT(doc.doctor_id) AS doctor_count
FROM Department d LEFT JOIN Doctor doc ON d.dept_id = doc.dept_id
GROUP BY d.dept_id, d.dept_name ORDER BY d.dept_id;
```

-- Q6: Treatment, patient and ward

```
SELECT CONCAT(p.first_name, ' ', p.last_name) AS patient, t.diagnosis,
       COALESCE(w.ward_name, 'Outpatient') AS ward
FROM Treatment t JOIN Appointment a ON t.appt_id = a.appt_id
JOIN Patient p ON a.patient_id = p.patient_id
```

LEFT JOIN Ward w ON t.ward_id = w.ward_id ORDER BY t.treatment_id;

-- Q7: Billing details and total

```
SELECT CONCAT(p.first_name, ' ', p.last_name) AS patient, b.payment_status,  
       (b.consultation_fee + b.medicine_charges + b.bed_charges) AS total_amount  
FROM Bill b JOIN Patient p ON b.patient_id = p.patient_id ORDER BY b.bill_id;
```

-- Q8: Bill aggregate statistics

```
SELECT COUNT(*) AS total_bills,  
       MAX(consultation_fee + medicine_charges + bed_charges) AS highest,  
       MIN(consultation_fee + medicine_charges + bed_charges) AS lowest,  
       ROUND(AVG(consultation_fee + medicine_charges + bed_charges),2) AS average  
FROM Bill;
```

-- Q9: Appointments per doctor

```
SELECT CONCAT(d.first_name, ' ', d.last_name) AS doctor, COUNT(a.appt_id) AS  
appointments  
FROM Doctor d LEFT JOIN Appointment a ON d.doctor_id = a.doctor_id  
GROUP BY d.doctor_id, d.first_name, d.last_name  
ORDER BY appointments DESC, doctor;
```

-- Q10: Doctors with more than one appointment

```
SELECT CONCAT(d.first_name, ' ', d.last_name) AS doctor, COUNT(*) AS total  
FROM Appointment a JOIN Doctor d ON a.doctor_id = d.doctor_id  
GROUP BY d.doctor_id, d.first_name, d.last_name HAVING COUNT(*) > 1;
```

-- Q11: Bills above average

```
SELECT bill_id, patient_id, (consultation_fee + medicine_charges + bed_charges) AS total  
FROM Bill WHERE (consultation_fee + medicine_charges + bed_charges) >  
(SELECT AVG(consultation_fee + medicine_charges + bed_charges) FROM Bill);
```

-- Q12: Latest appointment for each patient

```
SELECT a.appt_id, a.patient_id, a.appt_date, a.status FROM Appointment a
WHERE a.appt_date = (SELECT MAX(a2.appt_date) FROM Appointment a2
WHERE a2.patient_id = a.patient_id) ORDER BY a.patient_id;
```

-- Q13: Patient treatment view

```
SELECT * FROM v_patient_treatment ORDER BY treatment_date;
```

-- Q14: Pending billing view

```
SELECT * FROM v_bill_summary WHERE payment_status = 'Pending';
```

-- Q15: Departments with a doctor or ward

```
SELECT d.dept_name FROM Department d JOIN Doctor doc ON d.dept_id = doc.dept_id
UNION
SELECT d.dept_name FROM Department d JOIN Ward w ON d.dept_id = w.dept_id
ORDER BY dept_name;
```

-- Q16: Departments with both doctors and wards

```
SELECT d.dept_name FROM Department d JOIN Doctor doc ON d.dept_id = doc.dept_id
INTERSECT
SELECT d.dept_name FROM Department d JOIN Ward w ON d.dept_id = w.dept_id
ORDER BY dept_name;
```

-- Q17: Departments with doctors but no ward

```
SELECT d.dept_name FROM Department d JOIN Doctor doc ON d.dept_id = doc.dept_id
EXCEPT
SELECT d.dept_name FROM Department d JOIN Ward w ON d.dept_id = w.dept_id
ORDER BY dept_name;
```